



Coco UART SHDL C Interface

Sensirion AG

Rev. 1.0.0 / 2025-11-11

1	Preface	1
2	Introduction	2
3	Key Features	3
4	Hardware	4
4.1	UART	4
5	Frame Definition	5
5.1	Type of Frames	5
5.1.1	MOSI Frame	5
5.1.2	MISO Frame	5
5.2	Start and Stop	5
5.3	Address Field	6
5.4	Command	6
5.5	Length	6
5.6	State	6
5.6.1	Execution Error Code	8
5.6.2	Device Error Flag	8
5.7	Data	8
5.8	Checksum	8
6	Protocol Definition	10
6.1	Transfer Types	10
6.1.1	Standard Transfer	10
6.1.2	Broadcast Transfer	10
6.2	Error Response	11
7	Protocol Timings	12
7.1	Inter byte Timeout	12
7.2	Slave Response Timeout	12
8	Data Types and Representation	13
8.1	Integer	13
8.2	Boolean	13
8.3	Float (32 bit single precision)	13
8.4	String	13

9	Commands	15
9.1	[0x00] Start Measurement Mode	15
9.1.1	Start Periodic Measurement Mode	15
9.2	[0x01] Stop Measurement	15
9.3	[0x03] Read Measurement	15
9.3.1	Read Measurement Values	15
9.4	[0x04] Read Buffered Values	16
9.4.1	Read Buffered Measurement Values	16
9.5	[0x91] Get/Set Baudrate	17
9.6	[0xD0] Device Information	17
9.6.1	Get Product Type	17
9.7	[0xD1] Get Version	18
9.8	[0x9F] Get Firmware Build	18
10	State Response Error Codes	19
11	Revision history	20

Preface

This document describes the SHDLc communication interface of the Sensirion Coco CO2 sensor.

Introduction

The intention of defining the SHDLC protocol was to have a single protocol for different Sensirion devices as mass flow controllers, meters, and other microcontroller devices. The protocol is based on serial communication and is easy to implement. It allows communication between a single master and several connected slave devices.

Key Features

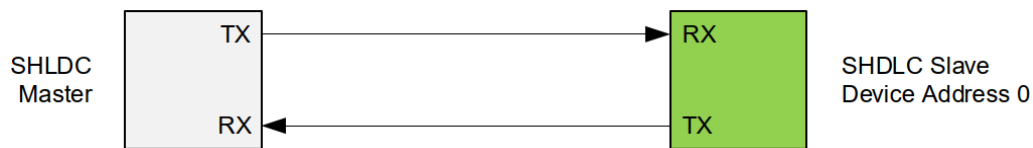
In the following, there are some of the SHDLC protocol key features:

- Based on an **byte orientated** hardware (like UART, . . .)
- **Half-duplex** system, which allows to use half-duplex hardware
- **Single Master/Multiple Slaves** protocol without the need for bus arbitration
- Allows up to **255 bytes payload data** for read or write transfers
- **Broadcasting** allows to safe bus load and synchronize devices

Hardware

The protocol is based on a byte oriented, bidirectional interface without hardware handshaking. Depending on the used hardware, it is possible to connect several devices on the same bus. The protocol uses half duplex communication, which allows to share the communication lines for Rx and Tx. An RS485/RS422 half duplex or UART hardware can be used as physical layer. Coco uses UART hardware.

4.1 UART



The UART as physical layer implements a point-to-point communication of the SHDLC protocol. This means only one SHDLC device can be connected to the master. This is ideal for short distance communication and allows to save an additional bus PHY. The UART uses the following settings:

- Baud rate as defined in data sheet (115'200 baud per default)
- 8 Data bits (LSb first)
- No Parity
- 1 Stop Bit

Frame Definition

Every transfer in SHDLC is based on MOSI (Master Out Slave In) and MISO (Master In Slave Out) frames. In the following chapter, the composition of the frames and the meaning of the different frame parts is shown.



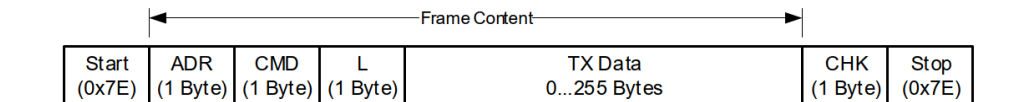
Note

In the following, the order of the bytes in frames is defined. The order of the bits inside a byte (MSb/LSb) is defined by the used hardware.

5.1 Type of Frames

5.1.1 MOSI Frame

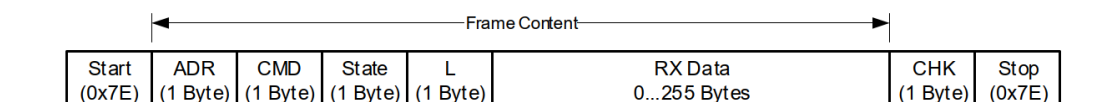
The MOSI frame is the message sent from master to slave as a request. Communication on the SHDLC bus is always initiated by the master. Therefore, this frame is always the begin of communication on the bus:



The begin and end of the frame is marked by start and stop bytes, which are unique bytes (0x7E) in the frame (guaranteed by byte stuffing, see Section 5.2). After the start byte, the address field defines the receiver of the frame, followed by the command which tells the slave what to do with the data. Before the payload data is sent, there is a length information (L) and after the data a checksum to guarantee the validity of the data.

5.1.2 MISO Frame

The MISO frame is the response from a slave device after reception of the MOSI frame. The SHDLC protocol guarantees an answer on any master request (except broadcast commands). This allows the master to ensure his request was correctly transferred and processed. The MISO frame looks as follows:



It is similar to the MOSI frame, except the additional state byte. In this field, the slave reports any problem during command execution and an additional general health information. In the address field, the slave fills in his own address and in the command field the command which was received and executed.

5.2 Start and Stop

Since SHDLC does not use hardware handshaking, we are using special bytes in the data stream to mark the begin and end of a frame:

- Start: 0x7E (01111110b)
- Stop: 0x7E (01111110b)

To make this working properly, we have to avoid this byte (0x7E) inside the frame. For this reason, byte stuffing is used, which means we are replacing 0x7E anywhere inside the frame with a sequence of two other bytes. This will also be done for Escape (0x7D), XON (0x11) and XOFF (0x13) bytes. The following table shows how the original data bytes are replaced in the sent data. All other data bytes are sent 1:1.

Original data byte	Transferred data bytes
0x7E	0x7D, 0x5E
0x7D	0x7D, 0x5D
0x11	0x7D, 0x31
0x13	0x7D, 0x33

5.3 Address Field

The address field in the MOSI frame (1 Byte) defines the receiver of the frame (slave device address). The address range is divided as follows:

- 0...254: Slave addresses
- 255: Broadcast address

In a MISO frame the address field contains the slave address (address of the slave which has sent the MISO frame).

5.4 Command

Typically (in a MOSI frame), this field contains the application command which defines for the specific application what to do with the given data. There are some reserved commands which are used for special frame transfers. In a MISO frame the slave will return the received and processed command in the CMD field.

The following table shows the organization of the available command space:

Command ID (Hex)	Size	Usage
0x00 .. 0x7F	128	Individual, device specific commands. The meaning and payload data of these commands depend on the type of device.
0x80 .. 0xEF	112	Common commands. The meaning and payload data of these commands are identical for all device types. However, not all commands are supported by each device type, i.e. the commands are optional.
0xF0 .. 0xFF	16	Special frame identifiers.

The size of the command is 1 byte.

5.5 Length

The length byte defines the number of transferred bytes in the data field (Rx or Tx). It is the length of the data field before byte stuffing, not the number of bytes which are transferred (because they contain stuffed bytes).

Example: The sender wants to transmit data [0xA7, 0xB4, 0x7E, 0x24]. Because of byte stuffing, it needs to transmit the stream [0xA7, 0xB4, 0x7D, 0x5E, 0x24]. The transmitted length information in this case is 0x04.

5.6 State

The MISO frame contains a state byte, which allows the master to detect errors on slave side. There are two kind of errors signaled in the state byte:

- Execution Errors
- Device Error Flag

The following table shows the composition of the state byte:

b7	b6	b5	b4	b3	b2	b1	b0
Device Error Flag	Execution error code						

5.6.1 Execution Error Code

The execution error code signals all errors occurred while processing the frame or executing the command. The errors are binary encoded in 7 bits with the following meaning:

Error Code	Meaning
0x00	No Error. Processing and execution of the command was successful.
0x01 ... 0x1F	Common error codes, which have the same meaning on every SHDLC device.
0x20 ... 0x7F	Error codes which are specific for every device. See device documentation.

5.6.2 Device Error Flag

This flag notifies the master that something occurred on the device during the background operation, which has nothing to do with the execution of the command (for example if the supply voltage is too low). If this flag is set, the master can read the device error state register with the "Read Device Status" command to check what happened exactly.

5.7 Data

The data has a usable size of [0...255] bytes (original data, before byte stuffing). The meaning of the data content is defined in the device's command reference.

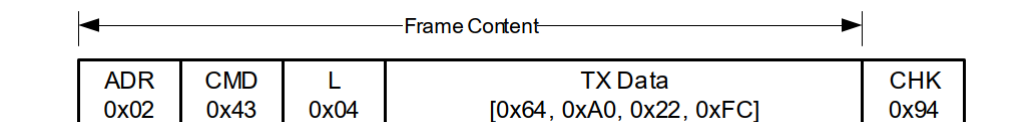
5.8 Checksum

The checksum is built before byte stuffing and checked after removing stuffed bytes from the frame. It is calculated as follows:

1. Sum all bytes between start and stop (without start and stop bytes)
2. Take the LSB of the result and invert it. This will be the checksum.

Example

Consider this MOSI frame (without start/stop and without byte stuffing):



The checksum calculates as follows:

ADR	0x02
CMD	0x43
L	0x04
Data 0	0x64
Data 1	0xA0
Data 2	0x22

Data 3	0xFC
Sum of all Bytes	0x26B
LSB of the Sum	0x6B
Inverted (=Checksum)	0x94

Protocol Definition

This chapter describes the frame communication protocol in a SHDLC system. There are some basic rules:

1. On every master request (MOSI frame), the slave will respond with a slave response (MISO frame). There are two exclusions where the slave will not send a response:
 - If the checksum of a MOSI frame did not match
 - If the MOSI frame was a broadcast
2. Between receiving a MOSI frame and sending slave response, the slave will not accept any other frame from master. In case of a broadcast, the master has to wait the specified command execution time.

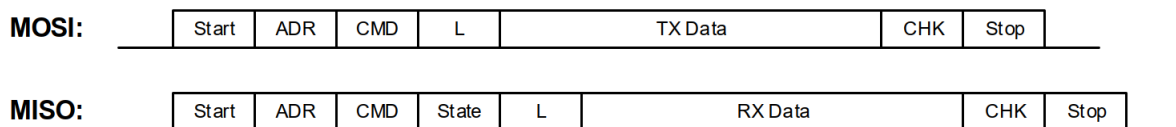
6.1 Transfer Types

By default, the master sends a standard frame which contains up to 255 bytes Tx data. This is called a standard frame transfer. Additionally there are some special frame transfers defined. They are marked with a special frame identifier in the CMD field of the frame. The following chapters describe the different transfer types.

6.1.1 Standard Transfer

In this transfer, the master initiates a transfer with a MOSI frame containing command ID and up to 255 bytes of Tx data. After executing the command, the slave will respond with a MISO frame containing state and up to 255 bytes of Rx data. The command field of the frame is used for an application command.

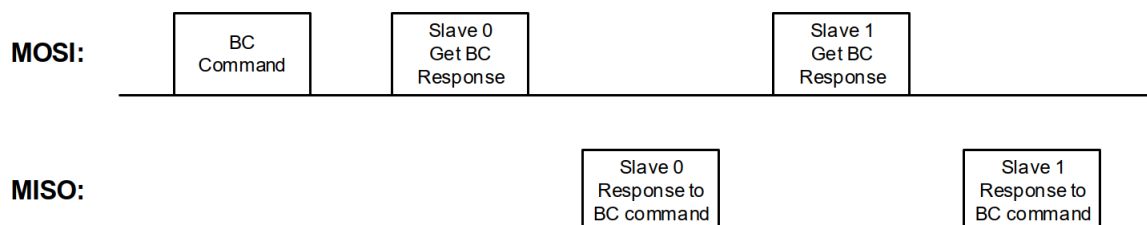
The transfer looks as follows:



6.1.2 Broadcast Transfer

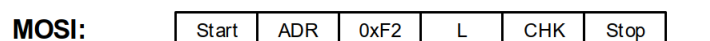
After sending a broadcast command (ADR = 0xFF), the slave executes the command but does not send the generated response. The “Get Broadcast Response” frame allows the master to get the slave response on a previously sent broadcast command.

The following shows an example with two slaves:



If the broadcast transfer is followed by a “Get Broadcast Response” frame, the slave will send the buffered answer. If any other frame is sent, the buffered response is discarded.

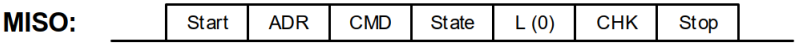
The frame to get the broadcast response (MOSI) looks as follows and has to be addressed to a specific slave device:



The slave answers with the same response as on an addressed command.

6.2 Error Response

In case of a command execution error, the device will return an error response. This response is transmitted without data (L=0). That means that a simple error response looks alike for any transfer type:



Protocol Timings

7.1 Inter byte Timeout

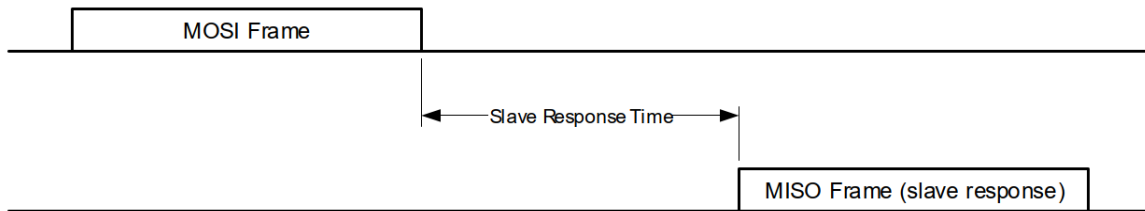
The inter byte time defines the time between two bytes in the same frame. After reception of a frame byte, the receiver waits for the next frame byte. This time is limited by the inter byte timeout. See the following timing diagram which defines the inter byte time:



The inter byte timeout is set to **200ms**. If a timeout occurs on slave side, the received data is discarded and no response frame will be sent to the master.

7.2 Slave Response Timeout

The slave response time is the time between the MOSI frame has left the master port and the begin of the reception of the MISO frame. This time is defined in the command reference.



The timeout should be at least **twice the specified maximum response time** of the corresponding command and in none real time systems it should not be smaller than **200ms** due to possible system side delays.

Data Types and Representation

The data in the frames is transmitted in big-endian order (MSB first).

8.1 Integer

Integers can be transmitted as signed or unsigned integers. If signed, use the two's complement. The following types of integers are known:

Integer Type	Size	Range
u8t	1 Byte	$0 \dots 2^8-1$
u16t	2 Byte	$0 \dots 2^{16}-1$
u32t	4 Byte	$0 \dots 2^{32}-1$
u64t	8 Byte	$0 \dots 2^{64}-1$
i8t	1 Byte	$-2^7 \dots 2^7-1$
i16t	2 Byte	$-2^{15} \dots 2^{15}-1$
i32t	4 Byte	$-2^{31} \dots 2^{31}-1$
i64t	8 Byte	$-2^{63} \dots 2^{63}-1$

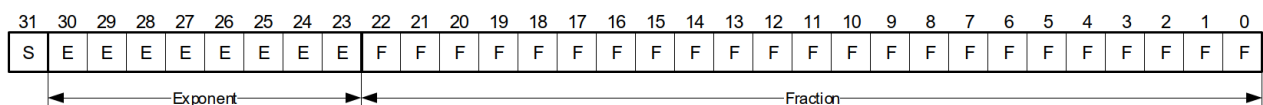
8.2 Boolean

A boolean is represented by 1 byte:

- False = 0
- True = 1...255

8.3 Float (32 bit single precision)

For floating-point representation, the IEEE 754 format is used which has the following structure:



Use the following coding to signal invalid float, positive or negative infinity:

Value	Coding (hex)
invalid float (NaN)	0xFFFFFFFF
+ infinity	0x7F800000
- infinity	0xFF800000

8.4 String

Strings are transferred as C-strings. This means in ASCII coding, one byte per character and terminated with a final null-character (0x00). The first letter will be sent first.

To make string handling as safe and robust as possible, it's highly recommended to follow these rules when implementing an SHDLG master:

- When sending strings to the slave (MOSI data), **always terminate the string with exactly one null-character (0x00)**. In case a fixed data length is specified for a particular string (e.g. if more MOSI parameters are following that string), fill all unused data bytes after the null-termination with 0x00 as well. But usually string parameters are specified with a variable data length, i.e. no such padding is needed.
- When receiving strings from the slave (MISO data), **parse the string up to the first null-character (0x00)** in the received data and **discard all the following data bytes** (if any). In case no null-character is contained in the whole frame data, **take the received data length into account to determine the string length**, i.e. abort string parsing after the last data byte to accept strings which are not null-terminated.

Commands

9.1 [0x00] Start Measurement Mode

9.1.1 Start Periodic Measurement Mode

Starts periodic measurement of CO₂, flow, pressure and temperature.

This command is only available in idle mode. If the device is already in any measure mode, a device error will be returned.

Start Periodic Measurement Mode		
Command ID	0x00	
Max. Response Time	50 ms	
Post Processing Time	0 ms	
MOSI Data	Byte #	Description
	0	Subcommand: uint8 Must be set to 0x01.
MISO Data	None	

9.2 [0x01] Stop Measurement

Stops the measurement and returns to idle mode.

The last measurement results can still be read out, if available.

If the device is already in idle mode, a device error will be returned.

Stop Measurement	
Command ID	0x01
Max. Response Time	100 ms
Post Processing Time	0 ms
MOSI Data	None
MISO Data	None

9.3 [0x03] Read Measurement

9.3.1 Read Measurement Values

Returns the last measured values. If no new data is available, the previous values will be returned again. If no data is available at all (no measurement running or immediately after starting the measurement), a device error will be returned.

This command does not remove the values from the buffer. In contrast, the command ``0x04 read buffered values`` removes the last measurement values from the buffer, such that they cannot be read again with this command!

Read Measurement Values		
Command ID	0x03	
Max. Response Time	50 ms	
Post Processing Time	0 ms	
MOSI Data	Byte #	Description
	0	Subcommand: uint8 Must be set to 0x00.
MISO Data	Byte #	Description

Read Measurement Values		
	0 ... 3	Counter: <code>uint32</code> The measurement counter is incremented for each measurement data set. The first measurement after starting the measurement has the counter value 0. The measurement counter overflows after value $(2^{32})-1$ and continues counting from 0.
	4 ... 5	Co2 Partial Pressure: <code>uint16</code> CO2 partial pressure in 0.1 mmHg. A value 0xFFFF indicates that no valid CO2 measurement is available.
	6 ... 7	Flow: <code>int16</code> Gas flow in (1/120) slm. A value 0x7FFF indicates that no valid flow measurement is available.
	8 ... 11	Pressure: <code>uint32</code> Pressure in (1/4096) hPa. A value 0xFFFF FFFF indicates that no valid pressure measurement is available.
	12 ... 13	Temperature: <code>int16</code> Temperature in (1/200) °C. A value 0x7FFF indicates that no valid temperature measurement is available.

9.4 [0x04] Read Buffered Values

9.4.1 Read Buffered Measurement Values

The device stores the last measurement values in a circular buffer. This command returns and clears the oldest values from the buffer.

If no data is available in the buffer, a device error will be returned. If the device is in idle mode, the last measurement values will be returned until the buffer is empty.

Read Buffered Measurement Values		
Command ID	0x04	
Max. Response Time	50 ms	
Post Processing Time	0 ms	
MOSI Data	Byte #	Description
	0	Subcommand: <code>uint8</code> Must be set to 0x00.
MISO Data	Byte #	Description
	0 ... 3	Counter: <code>uint32</code> The measurement counter is incremented for each measurement data set. The first measurement after starting the measurement has the counter value 0. The measurement counter overflows after $(2^{32})-1$ measurements and continues counting from 0. The returned counter value identifies the oldest measurement data set contained in the response. The measurement counter can be used to identify missing samples due to an overflow of the circular buffer (e.g. because the read-out rate is too low).
	4 ... 7	Number Of Remaining Values: <code>uint32</code> Number of measurement data sets remaining in the circular buffer after the read-out.
	8	Format: <code>uint8</code> Format version of the returned measurement data sets. The format version is defined by the firmware version and can change in future firmware versions.

Read Buffered Measurement Values		
	9 ... n	Data: bytearray Measurement data sets. The interpretation of the data depends on the format. The number of returned data sets is limited by the maximum payload size of the SHDLCL frame and the amount of available data in the circular buffer.

9.5 [0x91] Get/Set Baudrate

Get Baudrate		
Description	Get the SHDLCL baudrate of the device.	
Command ID	0x91	
Max. Response Time	50 ms	
Post Processing Time	0 ms	
MOSI Data	None	
MISO Data	Byte #	Description
	0 ... 3	Baudrate: uint32 Baudrate in bit/s.

Set Baudrate		
Description	Set the SHDLCL baudrate of the device.	
	 Note The baudrate is stored in the volatile memory of the device and is reset to the default baudrate after a device reset!	
Command ID	0x91	
Max. Response Time	50 ms	
Post Processing Time	0 ms	
MOSI Data	Byte #	Description
	0 ... 3	Baudrate: uint32 The new baudrate. See device documentation for a list of supported baudrates.
MISO Data	None	

9.6 [0xD0] Device Information

9.6.1 Get Product Type

Gets the product type from the device.

Get Product Type		
Command ID	0xD0	
Max. Response Time	50 ms	
Post Processing Time	0 ms	
MOSI Data	Byte #	Description
	0	Subcommand: uint8 Must be set to 0x00.
MISO Data	Byte #	Description
	0 ... 8	Product Type: string<9> String containing the product type.

9.7 [0xD1] Get Version

Gets the version information for the hardware, firmware and SHDLC protocol.

Get Version		
Command ID	0xD1	
Max. Response Time	50 ms	
Post Processing Time	0 ms	
MOSI Data	None	
MISO Data	Byte #	Description
	0	Firmware Major: <code>uint8</code> Firmware major version number.
	1	Firmware Minor: <code>uint8</code> Firmware minor version number.
	2	Firmware Develop Flag: <code>bool</code> Firmware develop flag. If it is set, the firmware is in development.
	3	Hardware Major: <code>uint8</code> Hardware major version number.
	4	Hardware Minor: <code>uint8</code> Hardware minor version number.
	5	Protocol Major: <code>uint8</code> Protocol major version number.
	6	Protocol Minor: <code>uint8</code> Protocol minor version number.

9.8 [0x9F] Get Firmware Build

Gets the identification of the firmware build on the device. For non-released firmware builds, this command returns more detailed information than the version numbers returned by the ``Get Version`` command.

Get Firmware Build		
Command ID	0x9F	
Max. Response Time	50 ms	
Post Processing Time	0 ms	
MOSI Data	None	
MISO Data	Byte #	Description
	0 ... n	Firmware Build: <code>string<0..32></code> String describing the firmware build.

State Response Error Codes

The state response error codes describe the meaning of the lower 7 bits of the 'state' byte which is returned in each MISO frame. See Section 5.6.1 for details.

Code		Name	Description
0	0x00	Success	Command successfully executed (no error occurred).
1	0x01	Data Size Error	Illegal data size of the MOSI frame. Either an invalid frame was sent or the firmware on the device does not support the requested feature.
2	0x02	Unknown Command Error	The device does not know this command. Either an invalid command ID was sent or the firmware on the device does not support it.
4	0x04	Parameter Error	A sent parameter is out of range. Check if you sent the correct command parameters and if the firmware on the device supports them.
32	0x20	No Measurement Data Available	No measurement data is available at the moment. Make sure the device is in measure mode.
67	0x43	Command Not Allowed In Current State	Command is not allowed in the current state.
112	0x70	Configuration Error	The configuration data either does not exist, is not compatible with the firmware (unsupported format or version), or the memory content is corrupt.
127	0x7F	Fatal Error	An error without specific error code occurred.

Revision history

Date	Revision	Changes
2025-11-11	1.0.0	Initial version